AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

Dept. of Computer Science
Faculty of Science and Technology

CSC2210: OBJECT ORIENTED PROGRAMMING 2

Spring 2024-2025

Section: A

Group No: 11

Project Report On

Janatar Haat Bazaar

Supervised By

Md. Hasibul Hasan

Submitted By:

Name	ID
[redacted]	[redacted]
[redacted] Md. Minhaj Rowfun Rabbi Anik	23-54110-3
[redacted]	[redacted]

Obtained Marks for CO2 and CO3 (Description given in the following page)

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)

Evaluation Criteria (CO2)	Total =	Evaluation Criteria (CO3)	Total =
Requirement fulfillment		Organization of the application	
Validation		Representation and Integration of Database	
Verification		Graphical User Interface	

CO2: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.					
Assessment Criteria	**Not Attended/ Incorrect (0)**	**Inadequate (1-2)**	**Average (3)**	**Good (4)**	**Excellent (5)**
Evaluation Criteria	**Evaluation Definition**				
Requirement fulfillment	Fails to demonstrate any understanding of real-life scenario-based project development or functional requirement identification. There is no attempt to depict a project or identify functional requirements accurately.	Demonstrates limited understanding of real-life scenario-based project development and functional requirement identification. The project depicted lacks coherence or relevance to real-life scenarios, and functional requirements are inaccurately identified or insufficiently described.	Presents a basic depiction of a real-life scenario-based project and identifies some functional requirements. However, the project lacks depth or complexity, and some functional requirements may be vaguely defined or missing key details.	Effectively demonstrates a realistic scenario-based project and accurately identifies most functional requirements. The project is well-developed with appropriate complexity, and functional requirements are clearly articulated with relevant details.	Exhibits an exceptional understanding of real-life scenario-based project development and accurately identifies all functional requirements. The project is meticulously developed with thorough attention to detail, reflecting a comprehensive understanding of Object-Oriented Programming project development activities.
Validation	Fails to demonstrate any understanding or implementation of validation forms in their system. There is no attempt to deal with data validation, and validation requirements are completely ignored or incorrectly applied.	Demonstrates limited understanding of validation forms and data validation techniques. While some attempt may be made to implement validation, it is incomplete or poorly executed, leading to inadequate handling of data validation.	Shows a basic understanding of validation forms and data validation techniques. They attempt to implement validation, but some aspects may be missing or incorrectly implemented, resulting in partial or inconsistent handling of data validation.	Effectively demonstrates the use of validation forms and implements data validation techniques. Validation is mostly accurate and comprehensive, ensuring the proper handling of data input and verification in the system.	Exhibits an exceptional understanding and implementation of validation forms and data validation techniques. Validation is meticulously implemented with thorough attention to detail, ensuring robust data validation procedures and contributing to the overall reliability and integrity of the system.

Verification	Fails to demonstrate any attempt to verify the system data or functional requirements. There is no evidence of understanding or implementation of verification processes, and data flow is not considered.	Demonstrates limited understanding of verification processes and data flow in the system. Verification attempts are incomplete or inaccurate, and there is insufficient consideration given to ensuring data integrity and functionality.	Shows a basic understanding of verification processes and attempts to verify system data. However, verification efforts may be inconsistent or lack thoroughness, and there may be gaps in ensuring proper functional requirements and data flow.	Identifies and verifies system data, ensuring proper functional requirements are met. Verification efforts are mostly accurate and thorough, with attention to ensuring data integrity and appropriate data flow within the system.	Exhibits an exceptional understanding of verification processes and meticulously verifies system data. Verification efforts are comprehensive and precise, with a keen focus on ensuring all functional requirements are met and maintaining proper data flow throughout the system.

CO3: Prepare and Explain a real life desktop based application synthesizing several component of C# along with development tools to adhere the given requirements.

Assessment Criteria	**Not Attended/ Incorrect (0)**	**Inadequate (1-2)**	**Average (3)**	**Good (4)**	**Excellent (5)**
Evaluation Criteria	**Evaluation Definition**				
Organization of the application	Fails to identify any suitable real time application or requirements for project development activities related to OOP.	Limited understanding about the project scopes and scenarios or identification of functional requirements.	Lacks depth or relevance to OOP project development activities and may contain inaccuracies. Real-life scenarios are mentioned, but the discussion lacks depth or clarity.	Consider and integrate the idea of several core aspects of the project along with relevance to real-life scenarios. Demonstrating a solid understanding of the application presentation.	Generalize and exhibits an exceptional understanding of project preparation according to a to real-life scenarios. Also contains proper and insightful identification of the system which is comprehensive and precise.
Representation and Integration of Database	Fails to identify and present any understanding or implementation of database. Also failed to integrate the data with the project itself.	Limited understanding of the database concepts or their proper way of using in a real time project. While some attempt may be made to implement but it is incomplete or poorly executed, leading to	Lacks depth or relevance to database integration with the application. Shows a basic understanding but some aspects may be missing or incorrectly implemented, resulting in partial or	Integrate the database with the forms properly and implements it with proper validation which is mostly accurate and comprehensive , ensuring the proper handling of data input and	Exhibits an exceptional understanding and implementation of database ensuring attention to detail, and robust data manipulation procedures and contributing to the overall clarity.

		inadequate design.	inconsistency. May lack proper normalization.	verification along with general normalization.	
Graphical User Interface	Fails to present or prepare GUI based application interfaces. There is no evidence of creating or integrating such things according to their usefulness.	Limited understanding of graphical user interfaces. Lack of design knowledge. Very poor attempt to make such things which are currently obsolete or can't be identified as coherent.	Shows a basic understanding of creating user interfaces. Most of them are interconnecte d but maybe some of them lack it. However, most of it can be described as user friendly.	Effectively identifies and meet the consider the simplicity. Design related works are mostly accurate and taken proper attention to ensuring a user-friendly coherent system.	Exhibits an exceptional work design following a high standard of simple and elegant work. Several controls and mechanism has been organized in a preferred way according to the coherent usage .

Table of Contents: **Page no.**

Supershop Management System (Janatar Haat Bazaar) Project Report

Chapter 01: Introduction

Welcome to the SuperShop Management System (Janatar Haat Bazaar) project report! This document outlines the development and functionality of a desktop application designed to streamline the management of Supershop named JanatarHaatBazaar. Our goal was to create a user-friendly and efficient system for both administrators and Employee, ensuring a smooth experience from Adding User creation to invoice Creation.

Chapter 02: User Story

Our SuperShop Management System caters to two primary user roles: **Administrators** and **Employee**. Here's a breakdown of their typical interactions with the system:

- **As an Administrator, I want to:**
  - Log in to an administrative dashboard.
  - Manage User, including adding new ones, updating their details , and deleting existing user.
  - Oversee user accounts, including auto password Generate and activating or deactivating user accounts.
  - Manage products, including creating new product, updating product details, and deleting Product.

- Create Customers, specifying details as name, phone number.
- Manage Sells Track quantities and sold by.
- Generate Invoice Print.
- View all registered User, Products, Invoice Reports And Can see Proper Create update, Delete.
- Log out from the system.

- **As a Employee, I want to:**
  - Log in to an Employee dashboard.
  - Add Sells and Can manage and also see all Sell Report.
  - Log out from the system.

Chapter 03a: ER Diagram

The Entity-Relationship (ER) diagram below illustrates the structure of our database, showing the entities (tables) and the relationships between them. This design adheres to **2NF (Second Normal Form)**, ensuring data integrity and minimizing redundancy.

Explanation of Entities and Relationships:

- **Role:** Stores specific Role details, (Pre Stored)
  - `roleID` (Primary Key,Auto-incrementing)
  - `roleName`
- **Users:** Stores user information.
  - `UserID` (Primary Key,Auto-incrementing)

    - fullName
    - Email (Unique)
    - Username (Unique)
    - Password
    - roleId
    - Mobile (unique)
    - Nid (Unique)
    - Address
      IsActive (Boolean to activate/deactivate accounts)
- **Product:** Stores Product details,
    - ProductID (Primary Key,Auto-incrementing)
    - ProductName
    - Quantity
    - manufactureDate
    - expiaryDate
    - categoryName
    - UnitPrice
    - Created_date
    - Is_active (Boolean to activate/deactivate accounts)
- **Customer:** Contains information about Customer.
    - CustomerID (Primary Key, Auto-incrementing)
    - CustomerName,
    - CustomerMobile
- **invoice:** Stores invoice details .
    - InvoiceID (Primary Key,Auto-incrementing)
    - CustomerId
    - invoiceDate
    - TotalAmount
    - discountAmount
    - finalAmount
    - createdBy
    - Created date
- **Invoice Items:** Stores invoice items details for invoice
    - InvoiceItemsID (Primary Key, Auto-incrementing)
    - invoiceId,
    - productId
    - Quantity
    - UnitPrice
    - TotalPrice

Chapter 03b: SQL Queries

Here are some key SQL queries and their purposes, demonstrating the **CRUD (Create, Read, Update, Delete)** operations.

1. Database Creation: The script begins by creating the JanatarHaatBazaar DB database.

```
Create Database JanatarHaatBazaar
```

2. Table Creation:

Below are examples of CREATE TABLE statements, showcasing the schema for key entities like Role,User,Product,customer,invoice,invoiceItems.

- Role Table:

```
CREATE TABLE role (
    id INT NOT NULL IDENTITY(1,1) PRIMARY KEY,
    role_name VARCHAR(20) NOT NULL
);
```

- Users Table:

```
CREATE TABLE users (
    id INT NOT NULL IDENTITY(1,1) PRIMARY KEY,
    full_name VARCHAR(32) NOT NULL,
    email VARCHAR(32) NOT NULL UNIQUE,
    username VARCHAR(20) NOT NULL UNIQUE,
    role_id INT NOT NULL,
    mobile VARCHAR(15) NOT NULL,
    password VARCHAR(255) NOT NULL,
    nid VARCHAR(19) NOT NULL UNIQUE,
    address VARCHAR(50) NOT NULL,
    created_date DATE,
    is_active BIT NOT NULL,
    FOREIGN KEY (role_id) REFERENCES role(id)
);
```

- Product Table:

```
CREATE TABLE product (
    id INT NOT NULL IDENTITY(1,1) PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    quantity INT NOT NULL,
    manufactureDate DATE NOT NULL,
    expiryDate DATE NOT NULL,
    categoryName VARCHAR(50) NOT NULL,
    unit_price DECIMAL(10, 2) NOT NULL,
    created_date DATE,
    is_active BIT NOT NULL
);
```

- Customer Table:

```
CREATE TABLE customer (
   id INT NOT NULL IDENTITY(1,1) PRIMARY KEY,
   name VARCHAR(50) NOT NULL,
   mobile VARCHAR(20) NOT NULL
);
```

- Invoice Table:

```
CREATE TABLE invoice (
    id INT NOT NULL IDENTITY(1,1) PRIMARY KEY,
    customerId INT NOT NULL,
    invoiceDate DATE NOT NULL DEFAULT GETDATE(),
    total_amount DECIMAL(10,2),
    discount_amount DECIMAL(10,2),
    final_amount DECIMAL(10,2),
    created_by INT,
    created_date DATE,
    FOREIGN KEY (customerId) REFERENCES customer(id)
);
```

- InvoiceItems Table:

```
CREATE TABLE invoice_items (
    id INT NOT NULL IDENTITY(1,1) PRIMARY KEY,
    invoice_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(10, 2) NOT NULL,
    total_price AS (quantity * unit_price),
    FOREIGN KEY (invoice_id) REFERENCES invoice(id),
    FOREIGN KEY (product_id) REFERENCES product(id)
);
```

- Seed Tables:

```
INSERT INTO role (role_name) VALUES ('Admin');
INSERT INTO role (role_name) VALUES ('Employee');

INSERT INTO users (
  full_name, email, username, role_id, mobile, password, nid, address, created_date, is_active
) VALUES ( 'Admin User', 'admin@gmail.com','admin',1,  '0123456789', 'admin123',
'1234567890', 'Dhaka Bangladesh',GETDATE(),1 );
```

2. Sql Query For Crud Operation:

1.User Module

- Insert User Query:

  ```
  INSERT INTO users (full_name, email, username, password, role_id, mobile, nid,
  address, created_date, is_active) VALUES ('{fullName}', '{email}', '{username}',
  '{password}', {roleId}, '{mobile}', '{nid}', '{address}', GETDATE(), {isActive})
  ```

- Database Validation Queries for User Creation:

  - Email validation: `SELECT email FROM users WHERE email = '{email}'`

  - Username validation: `SELECT username FROM users WHERE username = '{username}'`

  - Mobile validation: `SELECT mobile FROM users WHERE mobile = '{mobile}'`

  - NID validation: `SELECT nid FROM users WHERE nid = '{nid}'`

- Read (Show All Users):

This operation is handled in the `UserControlAdmin` Class for fetching user's data from DB with role name.

- Show All Users:

  SELECT u.id, u.full_name, u.email, u.username, r.role_name as role_name, u.mobile, u.nid, u.address, u.created_date, u.is_active

  FROM users u

  LEFT JOIN role r ON u.role_id = r.id

  ORDER BY u.id

- Search User Data:

  SELECT u.id, u.full_name, u.email, u.username, r.role_name as role_name, u.mobile, u.nid, u.address, u.created_date, u.is_active

  FROM users u

  LEFT JOIN role r ON u.role_id = r.id

  WHERE u.full_name LIKE '%{searchName}%'

  ORDER BY u.id

- Update (Update User):

  This operation is handled in the `FormEditUser` Class.

- Update User Query:

  UPDATE users

  SET full_name = '{fullName}', email = '{email}', username = '{username}', password = '{password}',

  role_id = {roleId}, mobile = '{mobile}', nid = '{nid}', address = '{address}', is_active = {isActive}

  WHERE id = {id}

- Database Validation Queries for User Update:

  - Email validation: `SELECT email FROM users WHERE email = '{email}' AND id != {UserId}`

  - Username validation: `SELECT username FROM users WHERE username = '{username}' AND id != {UserId}`

  - Mobile validation: `SELECT mobile FROM users WHERE mobile = '{mobile}' AND id != {UserId}`

  - NID validation: `SELECT nid FROM users WHERE nid = '{nid}' AND id != {UserId}`

- Get User by ID:

  SELECT * FROM users WHERE id = {UserId}

- Delete (Delete User):

  This operation is handled in the `UserControlAdmin` Class.

- Delete User Query:

  DELETE FROM users WHERE Id = '{id}'

2. AUTHENTICATION MODULE

- Login Operations

  This operation is handled in the `FormLogin` Class.

- User Authentication:

  SELECT u.id, u.username, u.password, u.full_name, r.role_name, u.is_active

  FROM users u

  INNER JOIN role r ON u.role_id = r.id

  WHERE u.username = '{username}'

  AND u.password = '{password}'

  AND r.role_name IN ('Admin', 'Employee')

  AND u.is_active = 1

- Check User Status:

  SELECT u.username, u.is_active, r.role_name

  FROM users u

  INNER JOIN role r ON u.role_id = r.id

  WHERE u.username = '{username}'

- Get User by Username (Admin/Employee Forms):

  SELECT * FROM users WHERE username = '{username}'

- Get User ID by Username:

  SELECT id FROM users WHERE username = '{username}'

3. PRODUCT MODULE

- Create (Add Product):

  This operation is handled in the `FormAddProduct` Class.

- Get Next Product ID:

  SELECT ISNULL(MAX(id), 0) + 1 FROM product

- Insert Product Query:

  INSERT INTO product (name, quantity, manufactureDate, expiryDate, categoryName, unit_price, created_date, is_active)

  VALUES('{name}', {quantity}, '{manufactureDate}', '{expiryDate}', '{categoryName}', {unitPrice}, '{createdDate}', {isActive})

- Read (Show All Products):

  This operation is handled in the `UserControlProduct` Class.

- Show All Products:

  SELECT * FROM product

- Search Product by Name:

  SELECT * FROM product WHERE name LIKE '%{searchName}%'

- Get Product by ID:

  SELECT * FROM product WHERE id = {id}

- Get Active Products for Sale:

  SELECT id, name, unit_price, quantity FROM product WHERE is_active = 1 AND quantity > 0

- Update (Update Product):

  This operation is handled in the `FormAddProduct` Class.

- Update Product Query:

  UPDATE product

  SET name = '{name}',

  quantity = {quantity},

  manufactureDate = '{manufactureDate}',

  expiryDate = '{expiryDate}',

  categoryName = '{categoryName}',

  unit_price = {unitPrice},

```
is_active = {isActive}
WHERE id = {productId}
```

- Update Product Quantity (After Sale):

```
UPDATE product
SET quantity = quantity - {quantity}
WHERE id = {productId}
```

- Restore Product Quantity (After Invoice Deletion/Edit):

```
UPDATE p
SET quantity = p.quantity + ii.quantity
FROM product p
INNER JOIN invoice_items ii ON p.id = ii.product_id
WHERE ii.invoice_id = {invoiceId}
```

- Delete (Delete Product):

  This operation is handled in the `UserControlProduct` Class.

- Delete Product Query:

  DELETE FROM product WHERE Id = '{productId}'

4. CUSTOMER MODULE

CRUD Operations for Customer Module

- Create (Add Customer):

  This operation is handled in the `FormAddCustomer` Class.

- Insert Customer Query:

  INSERT INTO customer (name, mobile) VALUES ('{name}', '{mobile}')

- Read (Show All Customers):

  This operation is handled in the `FormAddSale` Class.

- Get All Customers for Sale:

SELECT id, name, mobile FROM customer

5. SALES/INVOICE MODULE

CRUD Operations for Sales/Invoice Module

- Create (Add Sale/Invoice):

  This operation is handled in the `FormAddSale` Class.

- Insert Invoice Query:

```
INSERT INTO invoice (customerId, invoiceDate, total_amount, discount_amount,
final_amount, created_by, created_date)
VALUES ({customerId}, '{invoiceDate}', {totalAmount}, {discountAmount},
{finalAmount}, {userId}, GETDATE());
SELECT SCOPE_IDENTITY();
```

- Insert Invoice Items Query:

```
INSERT INTO invoice_items (invoice_id, product_id, quantity, unit_price)
VALUES ({invoiceId}, {productId}, {quantity}, {unitPrice})
```

- Read (Sales Reports)

  This operation is handled in the `FormSalesReport` Class.

- Get Sales Report with Date Range:

```
SELECT
  i.id as 'Invoice ID',
  c.name as 'Customer Name',
  c.mobile as 'Mobile',
  i.invoiceDate as 'Invoice Date',
  i.total_amount as 'Total Amount',
  i.discount_amount as 'Discount',
  i.final_amount as 'Final Amount',
  u.full_name as 'Created By'
FROM invoice i
INNER JOIN customer c ON i.customerId = c.id
LEFT JOIN users u ON i.created_by = u.id
```

```
WHERE i.invoiceDate BETWEEN '{fromDate}' AND '{toDate}'
ORDER BY i.invoiceDate DESC, i.id DESC
```

- Get Sales Summary Statistics:

```
SELECT
    COUNT(*) as TotalInvoices,
    ISNULL(SUM(total_amount), 0) as TotalSales,
    ISNULL(SUM(discount_amount), 0) as TotalDiscount,
    ISNULL(SUM(final_amount), 0) as NetSales,
    ISNULL(AVG(final_amount), 0) as AverageSale
FROM invoice
WHERE invoiceDate BETWEEN '{fromDate}' AND '{toDate}'
```

- Get Invoice Details by Invoice ID:

```
SELECT
    p.name as 'Product Name',
    ii.unit_price as 'Unit Price',
    ii.quantity as 'Quantity',
    ii.total_price as 'Total Price'
FROM invoice_items ii
INNER JOIN product p ON ii.product_id = p.id
WHERE ii.invoice_id = {invoiceId}
```

- Read (Invoice Print):

  This operation is handled in the `FormInvoicePrint` Class.

- Get Invoice Header Information:

```
SELECT
    i.id,
    i.invoiceDate,
    i.total_amount,
    i.discount_amount,
    i.final_amount,
    c.name as customer_name,
    c.mobile as customer_mobile,
    u.full_name as created_by
```

FROM invoice i

INNER JOIN customer c ON i.customerId = c.id

LEFT JOIN users u ON i.created_by = u.id

WHERE i.id = {invoiceId}

- Get Invoice Items for Printing:

```
SELECT
   p.name,
   ii.quantity,
   ii.unit_price,
   ii.total_price
FROM invoice_items ii
INNER JOIN product p ON ii.product_id = p.id
WHERE ii.invoice_id = {invoiceId}
```

- Update (Edit Invoice):

  This operation is handled in the `FormAddSale` Class (Edit Mode).

- Get Invoice Data for Editing:

```
SELECT i.customerId, i.invoiceDate, i.discount_amount, i.created_by, c.name as customer_name
FROM invoice i
INNER JOIN customer c ON i.customerId = c.id
WHERE i.id = {invoiceId}
```

- Get Invoice Items for Editing:

```
SELECT ii.product_id, p.name as product_name, ii.unit_price, ii.quantity,
     (ii.unit_price * ii.quantity) as total_price
FROM invoice_items ii
INNER JOIN product p ON ii.product_id = p.id
WHERE ii.invoice_id = {invoiceId}
```

- Update Invoice Query:

```
UPDATE invoice
SET customerId = {customerId},
```

```
    invoiceDate = '{invoiceDate}',
    total_amount = {totalAmount},
    discount_amount = {discountAmount},
    final_amount = {finalAmount}
WHERE id = {invoiceId}
```

- Delete (Delete Invoice):

  This operation is handled in the `FormSalesReport` Class.

- Delete Invoice Items:

  ```
  DELETE FROM invoice_items WHERE invoice_id = {invoiceId}
  ```

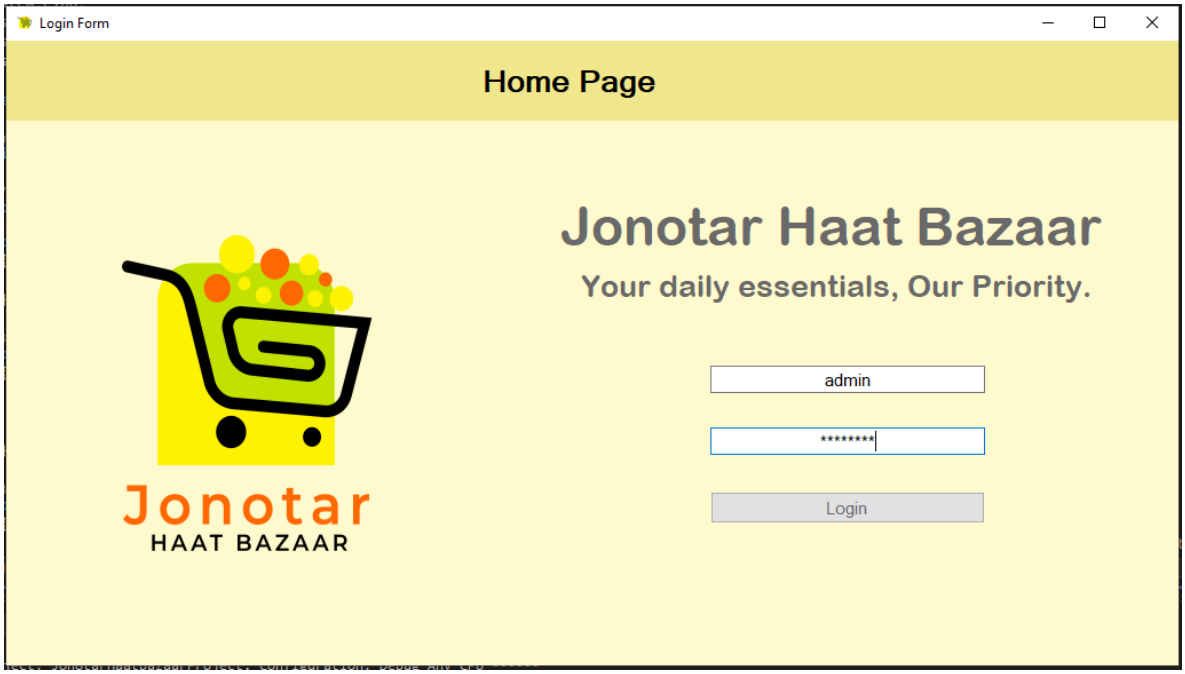
- Delete Invoice:

  ```
  DELETE FROM invoice WHERE id = {invoiceId}
  ```

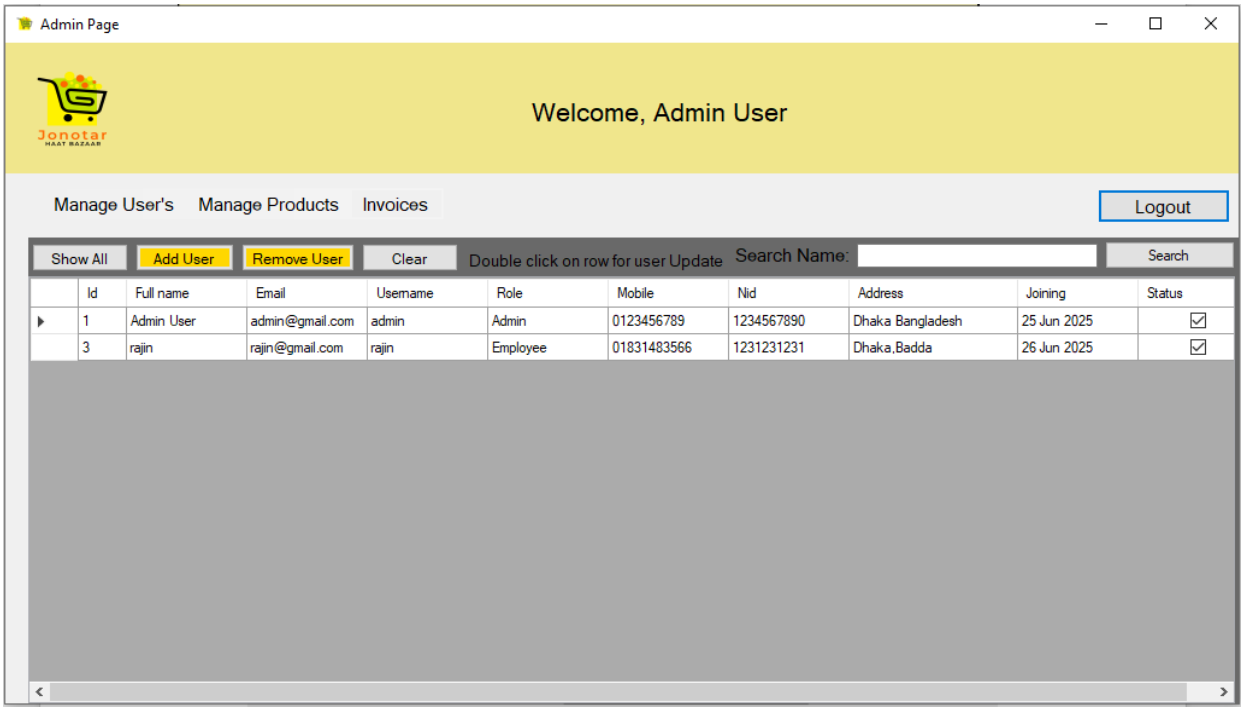
Chapter 04: Screenshots

(As I cannot directly embed interactive screenshots, I will describe what each screenshot would represent, based on the provided Designer.cs files and the project structure.)

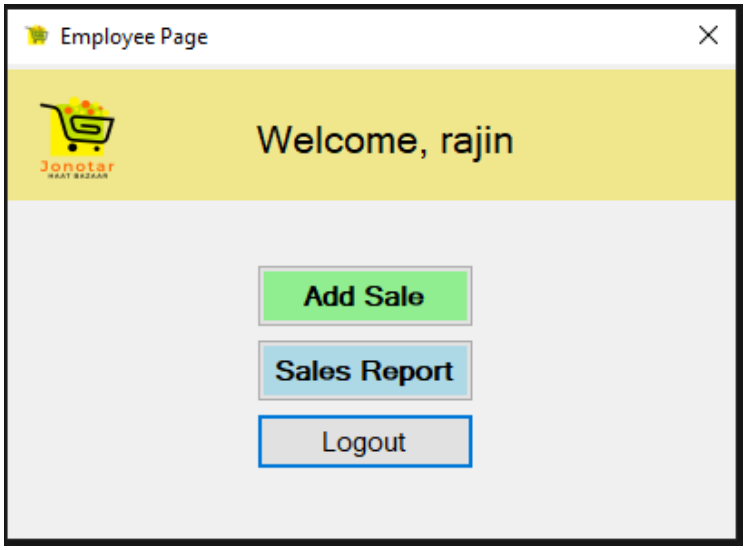
1. Login Form :This screenshot would display the initial entry point of the application.



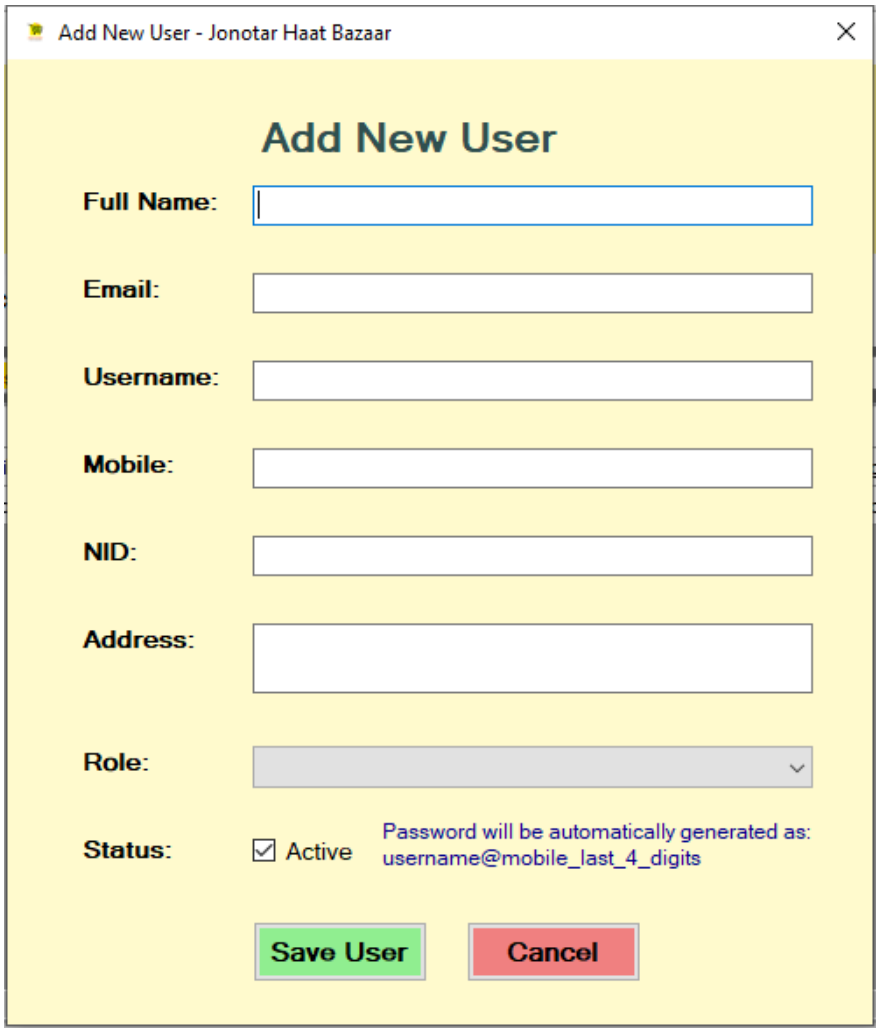
2. After Admin login it will show User Details table:



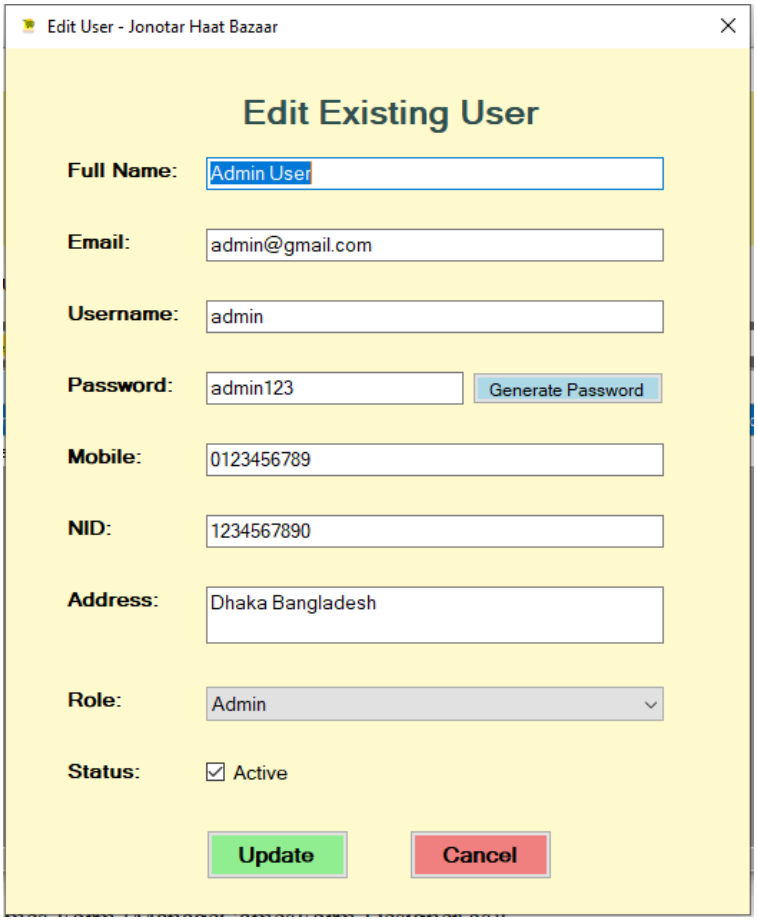
3. After employee Login Showing Dashboard.



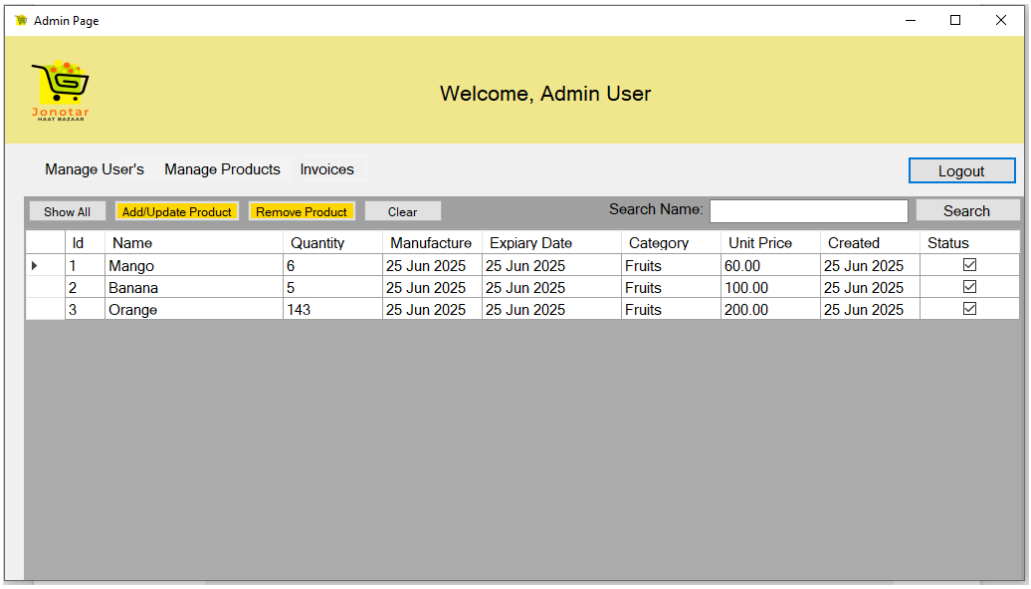
4. Add New User Form:



5. Update User Form:

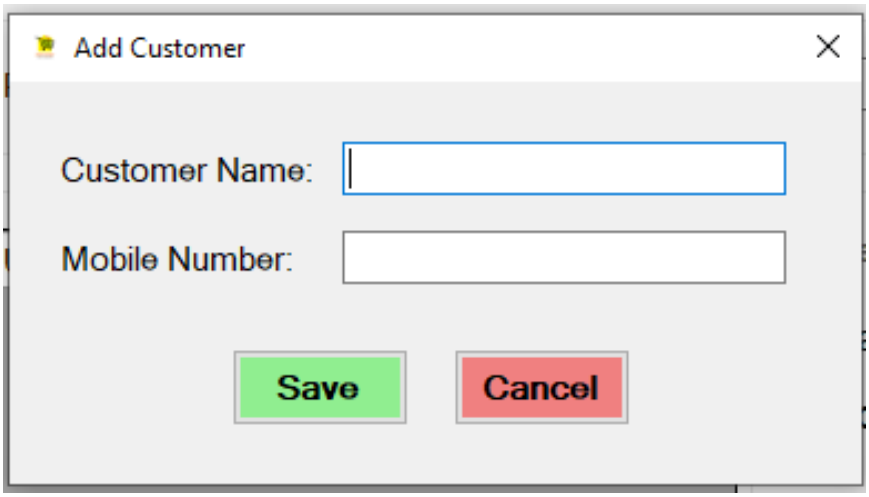


6. Product Datatables:

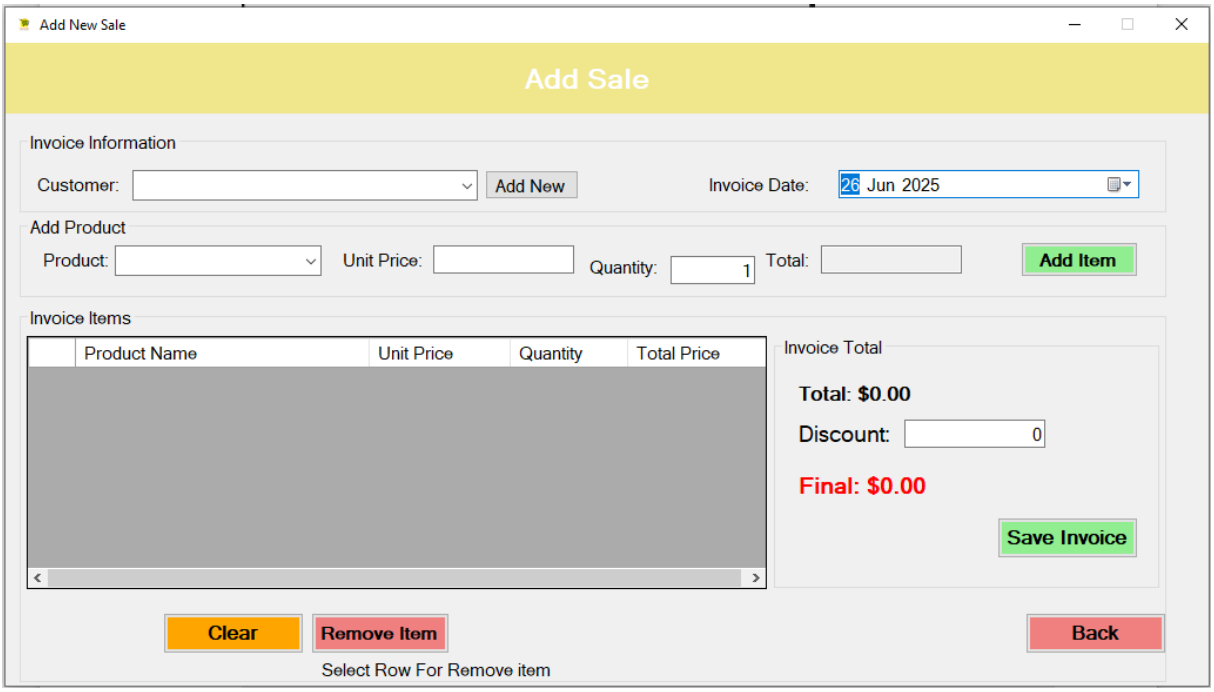


7. Add/update Product Form:

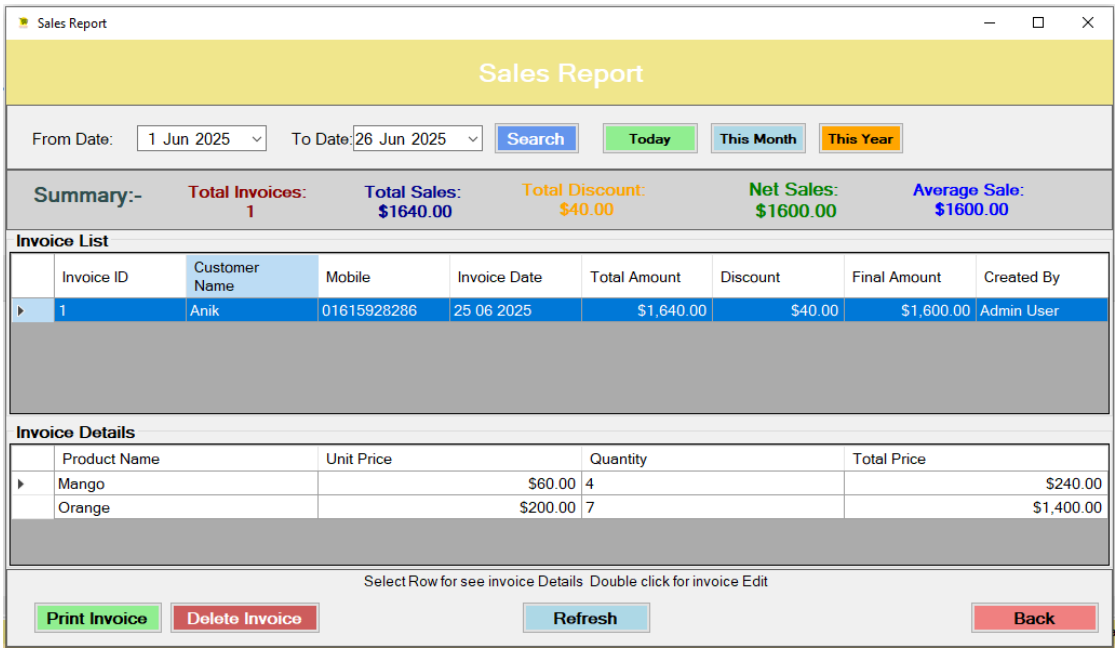
8. Add Customer Box:



9. Add/update Sell Form:



10. All Sells Report Window



11. Print Window:

This report provides a comprehensive overview of our Supershop Management System, covering its introduction, user stories, database design, and visual interface. If you have any further questions or need more details on specific sections, feel free to ask!

References

- For mssql 2012 Query: Link

  (learn.microsoft.com/en-us/dotnet/api/system.drawing.printing.printdocument.print?view=windowsdesktop-9.0)

- For Base String Class Method: Link

  (learn.microsoft.com/en-us/dotnet/api/system.string?view=net-9.0)

- For invoice Print: Link

  (learn.microsoft.com/en-us/dotnet/api/system.drawing.printing.printdocument.print?view=windowsdesktop-9.0)